Maze Escape Assignment Report

Student: Boranbayeva Zhansaya
Student ID: 254448
Group: SE-2532
Maze: 10 rows × 8 columns

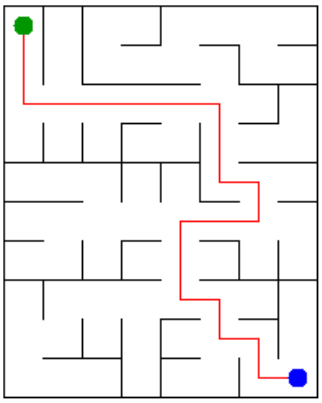
Program output

Broken: 79
Kept: 63
Last broken wall: 123
MST cost: 26435
Heaviest broken wall: 787
Path length: 20
DFS visited: 46
Dead ends: 30
Zones: 8
Largest zone: 15
Is DAG: 0
Flow stages: 6
Fastest route cost: 8

Validation

The project compiles, all 14 DSU checks pass, and the tiny, small and dag examples match all 13 expected output lines. The personal input files were also verified against the variant generated from Student ID 254448.

Maze visualization



Questions

Mission 0. What do find and union do, and what makes them fast in your code?

find returns the root representing the group that contains a cell, and union joins two different groups, returning false when they are already connected. Path compression makes every cell on a searched path point directly to the root, while union by size attaches the smaller tree to the larger one. Together, these techniques give an amortized O(α(n)) cost per find or union operation; connected compares roots, and count returns the stored number of groups.

Mission 1. Why is Broken always R*C - 1, and Kept always (R-1)*(C-1)?

A wall is broken only when it joins two different components, so each accepted wall reduces the component count by one and never creates a cycle. Starting with RC separate cells and ending with one connected component therefore requires RC − 1 broken walls. The full rectangular grid has R(C − 1) + C(R − 1) inner walls, so the number kept is (R − 1)(C − 1).

Broken=10x8-1=79
Kept=10x7+8x9-79=63

Mission 2. My MST cost from Kruskal and from Prim, and why they are equal:

Kruskal considers walls in increasing cost order and uses DSU to avoid cycles; Prim grows a connected tree by selecting the cheapest edge to an unvisited cell from a priority queue. Both algorithms find a minimum spanning tree of the same connected graph, and both give a total cost of 26435. All wall costs are different, so this minimum spanning tree is unique; its heaviest selected wall costs 787.

Mission 3. Why is there exactly one path between two cells? How did you avoid a stack overflow on a huge maze?

The maze from Mission 1 is connected and has no cycles, so it is a tree and has exactly one simple path between any two cells. DFS uses explicit arrays for the current path and the next direction to try, avoiding recursive calls and stack overflow on deep searches. It tries up, right, down and left, marks 46 cells by the time it reaches the exit, returns a path of 20 moves and counts 30 cells with exactly one open neighbour.

Mission 4. Why does the second pass of Kosaraju go on the reversed graph? Why do the slides have no cycle exactly when Zones equals the number of cells?

The first DFS records finishing times, and the second processes cells in decreasing finishing order on the reversed graph. This ordering allows each second-pass search to collect one strongly connected component without entering another unassigned component, and numbers the resulting zones in topological order. Since the input has no self-loops, it is acyclic exactly when every zone contains one cell; here there are 8 zones for 80 cells, so Is DAG is 0.

Mission 5. Why is it enough to go through the zones in the order of their numbers? What does this pass cost?

Every edge between different zones goes from a smaller zone number to a larger one, so all predecessors have been processed when a zone is reached. A single pass updates the longest chain and the shortest distance from the start zone; travel within a zone is free, and parallel transitions keep their minimum cost. With k zones and e distinct interzone edges, this pass takes O(k + e) time; building the graph from m slides takes expected O(m + k + e) time with hash maps, producing 6 flow stages and route cost 8.

Timing results

Bench runs each algorithm five times per size on generated grid graphs. All values below are milliseconds, rounded as printed by Bench; the reported average is calculated from the unrounded measurements.

Maze	Algorithm	Five runs in ms	Average ms
100 × 100	Kruskal	21, 9, 20, 21, 21	18
100 × 100	Prim	21, 10, 7, 7, 6	10
300 × 300	Kruskal	97, 72, 72, 60, 61	72
300 × 300	Prim	80, 89, 91, 88, 94	88
1000 × 1000	Kruskal	1012, 960, 953, 1204, 1001	1026
1000 × 1000	Prim	1914, 2006, 1986, 2466, 2216	2118

As the grid grows from 100 × 100 to 1000 × 1000, the average rises from 18 to 1026 ms for Kruskal and from 10 to 2118 ms for Prim. Prim is faster on the smallest grid, while Kruskal is faster on the two larger grids. Both implementations have an O(n log n) bound on these grid graphs; JVM warm-up, garbage collection and allocation costs can affect the measured times.

Time and space complexity

n denotes cells; m denotes relevant walls or slides; k denotes zones; e denotes distinct interzone edges. Space includes working structures and constructed adjacency lists, excluding input storage and images.

Algorithm	Time	Space	Assumptions
DSU find and union	$O(\alpha(n))$	$O(n)$	Amortized per operation; DSU initialization takes O(n).
Build maze	$O(n + m\alpha(n))$	$O(n)$	Walls processed in file order; DSU and selected edges.
Kruskal	$O(m \log m)$	$O(n + m)$	Connected grid; sort wall indices, then DSU operations.
Prim	$O(n + m \log m)$	$O(n + m)$	Lazy edge queue; maximum cell degree 4.
DFS escape	$O(n + m)$	$O(n + m)$	Explicit stack and adjacency lists; open maze is a tree.
Kosaraju	$O(n + m)$	$O(n + m)$	Two iterative DFS passes and both edge directions.
Zone routes	$O(m + k + e)$	$O(k + e)$	Expected hash-map construction. DAG pass: O(k + e).

α(n) is the inverse Ackermann function. On the rectangular grids, m = O(n), so the Kruskal and Prim bounds reduce to O(n log n). The route bound assumes zoneOf has already been computed.

AI use

I used ChatGPT to get a brief explanation of the assignment, understand the code, and generate draft code examples for the implementation.